

11 — Developer & Maintenance Documentation

Interface reference for every microservice (AWS Lambda), in a Boto3-style format: **parameters, formats, and results**. This is the contract a maintainer needs to call or modify each service. The machine-readable version is [../backend/api/openapi.yaml](#); the prose companion is [../docs/api.md](#).

Conventions

- **Transport:** JSON over HTTPS through API Gateway HTTP API. Request bodies are `application/json`. Errors return `{ "message": "..."}` with the listed status code.
- **Authentication & authorization:** the **write** services (`add_host`, `update_host`, `delete_host`, `add_guest`, `update_guest`, `delete_guest`, `set_tables`, `generate_seating`, `send_invitation`) and `list_hosts` require an `Authorization: Bearer <Cognito access token>` header. The Lambda confirms the caller **owns** the event (token email == `host_email`) or is an **admin** — a shared `_common.require_owner` helper bundled into each function — otherwise it returns `401` (missing/invalid token) or `403` (not the owner/admin). The **read** services (`get_host`, `get_guests`, `rsvp_guest`) are **public**. Direct SDK invokes with no API Gateway request context (the seed script) are trusted server-side calls and bypass the check.
- **Identifiers:** a host/event is identified by `host_email`; a guest by (`host_email`, `guest_email`).
- **Numbers as strings:** read endpoints serialize DynamoDB `Decimal` values as strings (`count`, `table`, `capacity`).
- **Source:** each service is one file in [../backend/lambdas](#); the Feature it implements is in its docstring.

Hosts

`add_host` — `POST /hosts` (Feature F06)

Create an event.

Parameter	Type	Required	Notes
<code>name</code>	string	yes	Host display name
<code>email</code>	string (email)	yes	Host email = partition key = Cognito username
<code>event_name</code>	string	yes	
<code>event_date</code>	string YYYY-MM-DD	yes	Must parse as a date
<code>event_location</code>	string	yes	

Returns: `201` the created host item (empty `guests/tables`, default `categories`). **Errors:** `400` validation, `401/403` not signed in / not the owner, `409` host already exists.

`get_host` — `GET /hosts?host_email=` (Feature F07)

Fetch an event summary.

Parameter	In	Type	Required
host_email	query	string (email)	yes

Returns: 200 { email, name, event_name, event_date, event_location, tables, categories, guest_count }. **Errors:** 400 missing/invalid host_email, 404 not found.

update_host — PUT /hosts (Feature F08)

Update event details / categories.

Parameter	Type	Required	Notes
host_email	string (email)	yes	Identifies the event
name	string	no	
event_name	string	no	
event_date	string YYYY-MM-DD	no	
event_location	string	no	
categories	string[]	no	Non-empty strings; duplicates removed case-insensitively

Returns: 200 success message. **Errors:** 400 nothing to update / invalid value, 401/403 not signed in / not the owner, 404 not found.

delete_host — DELETE /hosts (Feature F09)

Delete an event and all its guests.

Parameter	Type	Required
email	string (email)	yes

Returns: 200 deleted. **Errors:** 401/403 not signed in / not the owner, 404 not found.

Guests

add_guest — POST /guests (Feature F11)

Parameter	Type	Required	Notes
host_email	string (email)	yes	
name	string	yes	
guest_email	string (email)	yes	Unique within the host

Parameter	Type	Required	Notes
<code>category</code>	string	no	
<code>count</code>	integer	no	Party size, default <code>1</code> (range 1–20)

Returns: `201` created (new guest starts `rsvp:"?"`, `table:null`). **Errors:** `400` validation, `401/403` not signed in / not the owner, `409` host missing or guest already exists.

`get_guests` — GET `/guests?host_email=` (Feature F12)

Parameter	In	Type	Required
<code>host_email</code>	query	string (email)	yes

Returns: `200` { `guests`: { `<guest_email>`: { `name`, `category`, `count`, `rsvp`, `table` } } }. **Errors:** `400` missing/invalid `host_email`, `404` not found.

`update_guest` — PUT `/guests` (Feature F13)

Parameter	Type	Required	Notes
<code>host_email</code>	string (email)	yes	
<code>guest_email</code>	string (email)	yes	
<code>name</code>	string	no	
<code>category</code>	string	no	
<code>rsvp</code>	string	no	<code>yes</code> / <code>no</code> / <code>?</code>
<code>count</code>	integer	no	Party size, 1–20
<code>table</code>	integer or null	no	<code>null</code> clears the seat

Returns: `200` updated. **Errors:** `400` nothing to update / invalid `count`, `401/403` not signed in / not the owner, `404` host or guest not found.

`delete_guest` — DELETE `/guests` (Feature F14)

Parameter	Type	Required
<code>host_email</code>	string (email)	yes
<code>guest_email</code>	string (email)	yes

Returns: `200` deleted. **Errors:** `401/403` not signed in / not the owner, `404` host or guest not found.

`rsvp_guest` — POST `/guests/rsvp` (Feature F15, public)

Used by the public RSVP page; no authentication.

Parameter	Type	Required	Notes
host_email	string (email)	yes	
guest_email	string (email)	yes	
rsvp	string	yes	yes / no / ?
count	integer	no	
song	string	no	

Returns: 200 updated. **Errors:** 400 validation, 404 host or guest not found.

Tables & seating

set_tables — POST /tables (Feature F10)

Replaces the table configuration.

Parameter	Type	Required	Notes
host_email	string (email)	yes	
tables	map { tableNumber: { capacity } }	yes	Non-empty; each capacity a positive integer

Returns: 200 saved. **Errors:** 400 validation, 401/403 not signed in / not the owner, 404 not found.

generate_seating — POST /seating (Feature F16)

Parameter	Type	Required
host_email	string (email)	yes

Returns: 200 { message, total_guests_seated, assignments: { <guest_email>:
tableNumber } }. **Errors:** 400 no tables defined / not enough seats, 401/403 not signed in / not the owner, 404 not found.

Invitations

send_invitation — POST /invitations/send (Feature F17)

Parameter	Type	Required	Notes
host_email	string (email)	yes	
site_url	string (URL)	yes	Base URL used to build each RSVP link
guest_email	string (email)	no	Omit to invite every guest
message	string	no	Optional custom note

Returns (single guest): `200 { status:"sent" }` or `202 { status:"pending" }` (guest must confirm the SNS subscription first). **Returns (all guests):** `200 { message, sent[], pending[], failed[] }`. **Errors:** `400` missing `host_email`, `401/403` not signed in / not the owner, `404` no guests / not found, `500` email service unavailable.

Admin

`list_hosts` — `GET /admin/hosts` (Feature F18, admin only)

Lists every event on the platform.

Parameter	In	Type	Required	Notes
<code>Authorization</code>	header	<code>Bearer <access token></code>	yes	Cognito access token

Authorization logic (inside the Lambda):

1. Read the bearer token from the `Authorization` header.
2. `cognito-idp:GetUser(AccessToken)` — validates the token (else `401`).
3. `cognito-idp:AdminListGroupsForUser(UserPoolId, Username)` — caller must be in the `admin` group (else `403`).

Returns: `200 { total, hosts: [ { email, name, event_name, event_date, event_location, guest_count } ] }` (sorted by date then email). **Errors:** `401` missing/invalid token, `403` not an admin, `500` pool/Cognito error.

Maintenance notes

- **Add a new route:** create `backend/lambda/<name>.py` with a `lambda_handler`, add `('<name>', '<METHOD>', '<path>')` to the `ROUTES` list in `backend/deploy/setup_aws.py`, document it in `openapi.yaml` and `docs/api.md`, then redeploy. `redploy_all.py` (and its teardown) iterate `ROUTES`, so the new function is created and removed automatically.
- **Change CORS:** edit the CORS block in `setup_aws.py` (currently allows `Content-Type`, `Authorization`).
- **Data shape:** all event data is one DynamoDB item per host; nested `guests/tables` maps. Avoid unbounded growth per item (DynamoDB 400 KB item limit) — for very large events this would need a child-item redesign (roadmap).
- **Region:** everything is `us-east-1`; keep new resources there.